

Smart Campus Lab & Resource Reservation System

A Concurrent, Object-Oriented Academic Facility Scheduling Engine

Course Code & Title:	CSE2006: Programming in Java
Project Category:	Academic Course Project (Programming in Java)
Student Name:	Rohan Chetty
Registration Number:	25BAI10510
Degree & Branch:	B.Tech - Computer Science and Engineering
Semester / Academic Year:	Winter Semester 2025–2026
System Version:	v2.4.0-LTS (Final Submission)
Submission Date:	September 2026

1. Introduction

In modern academic institutions like VIT Bhopal University, engineering students and research scholars rely extensively on high-value laboratory facilities. These environments range from High-Performance Computing (HPC) AI clusters equipped with NVIDIA GPUs to specialized Internet of Things (IoT) testbeds, FPGA development stations, and Digital Storage Oscilloscopes. Under peak academic windows, uncoordinated access causes severe scheduling contention, double-booking, and inequitable facility distribution.

The **Smart Campus Lab & Resource Reservation System** is an original, object-oriented software application engineered in Java SE to automate and safeguard the facility reservation lifecycle. By implementing fair ReentrantLock mutex synchronization, atomic state persistence, and role-based access control, the system ensures zero-conflict resource allocation across interactive command-line and graphical desktop interfaces.

2. Problem Statement

Academic laboratory scheduling faces several critical failure modes under standard campus load:

- **Double-Booking & Race Conditions:** Simultaneous reservation attempts by multiple students for the same time slot frequently produce conflicting confirmations due to lack of thread-safe synchronization.
- **Monopolization & Quota Deficits:** Naive scheduling allows individual users to hoard scarce facilities for days, denying equitable access to peers.
- **Maintenance Blindspots:** Unannounced equipment recalibration or hardware faults lead to scheduled bookings on unusable hardware.
- **Operational Opacity:** Campus administrators lack real-time visibility into peak usage hours, lab utilization rates, and booking histories.

3. Functional Requirements

Module	Key Capabilities	Target User Tier
User Authentication & RBAC	Credential verification, session state management, quota enforcement (Student: 3, Faculty: 10, Admin: 99).	All Users
Campus Facility Catalog	Taxonomy of LabRooms (capacities, OS, GPU cluster, AV) and specialized Equipment (serial, condition rating).	Students, Faculty
Concurrent Reservation Engine	Thread-safe slot locking (ReentrantLock), double-booking prevention, 5 standard daily academic slots.	Students, Faculty
Admin Facility Governance	Maintenance mode toggle, decommission status, conflict cancellation, utilization analytics.	Lab Administrators

4. Non-Functional Requirements

- **Concurrency & Mutual Exclusion:** Absolute elimination of race conditions on contested slots via fair ReentrantLock synchronization.
- **Transaction Latency:** Booking confirmation, cancellation, and matrix generation under 15ms execution latency.

- **Data Persistence:** Atomic file persistence across system restarts in structured data/*.csv records.
- **Usability & Dual Interface:** Cross-platform ANSI terminal CLI accompanied by an interactive Swing desktop GUI.

5. System Architecture

The application is architected across four cohesive layers: Presentation (CLI/GUI), Application Service (Auth, Resource, Reservation, Analytics), Concurrency & Domain Core (SlotLockManager, User/Resource OOP hierarchies), and Persistence (FileDataStorage).

LAYER	COMPONENTS / CLASSES	RESPONSIBILITY
Presentation	ConsoleUI, CampusReserveGUI	Dual interface: ANSI interactive console and Java Swing desktop application.
Application Service	AuthService, ResourceService, ReservationService, AnalyticsService	Business rules, quota checks, session state, utilization aggregation.
Concurrency & Domain	SlotLockManager, User (Student/Faculty/Admin), Resource (LabRoom/Equipment)	Fine-grained ReentrantLock mutex registry, polymorphic OOP models.
Persistence Layer	DataStorage (interface), FileDataStorage (CSV persistence)	Atomic file I/O in data/ with thread-safe ConcurrentHashMaps.

6. Design Decisions & Technical Rationale

1. Fine-Grained Per-Slot Locks: Instead of synchronizing the entire reservation service—which would force a student booking an IoT bench to block another student reserving an AI lab—we maintain a `ConcurrentHashMap<String, ReentrantLock>` keyed by `[resourceId#date#slotId]`. This ensures maximum concurrent throughput with strict mutual exclusion on contending slots.

2. Polymorphic Role Quotas: Quotas are calculated via dynamic dispatch on abstract methods (`user.getMaxAllowedBookings()`). This eliminates brittle instanceof switch statements, upholding the Open-Closed Principle (OCP).

3. Zero-Dependency Persistent CSV Store: The application provides complete standalone portability without requiring MySQL or Docker configurations, running seamlessly on any JDK 17+ environment.

7. Implementation Details & Code Structure

The project consists of **18 modular Java classes** across 7 specialized packages, strictly adhering to Java enterprise package naming conventions:

- `com.campus.reserve.config.CampusConfig`: Centralized institutional parameters and campus branding.
- `com.campus.reserve.model.*`: User hierarchy (User, Student, Faculty, LabAdmin, UserRole), Resource hierarchy (Resource, LabRoom, Equipment, ResourceStatus), and Scheduling records (TimeSlot, Reservation, ReservationStatus).
- `com.campus.reserve.concurrency.*`: ReentrantLock coordinator (SlotLockManager), task wrapper (ConcurrentBookingWorker), and outcome telemetry (ReservationResult).
- `com.campus.reserve.storage.*`: Repository contract (DataStorage) and atomic file engine (FileDataStorage).
- `com.campus.reserve.service.*`: Transactional controllers (AuthService, ResourceService, ReservationService, AnalyticsService).

- `com.campus.reserve.ui.*`: Terminal interface (ConsoleUI) and Swing desktop app (CampusReserveGUI).
- `com.campus.reserve.test.ReservationTestSuite`: Automated unit & multi-threaded race condition verification.
- `com.campus.reserve.Main`: Multi-mode CLI, GUI, and test harness bootloader.

8. Testing Approach & Verification Results

The built-in verification suite (ReservationTestSuite) executes automated assertions across four critical functional pillars. Below is the verified test report output:

```
=====
ACADEMIC EVALUATION TEST SUITE: CSE2006 JAVA PROJECT
System: Smart Campus Lab & Resource Reservation System
=====

>>> [1/4] Running OOP Polymorphism & Quota Tests...
[PASS] Student Max Quota      - Student quota must be 3
[PASS] Faculty Max Quota     - Faculty quota must be 10
[PASS] Admin Override Check  - Admin can override maintenance
[PASS] Student Override Check - Student cannot override maintenance

>>> [2/4] Running Resource Hierarchy & Specs Tests...
[PASS] LabRoom Polymorphic Type - LabRoom correctly identifies as Laboratory Room
[PASS] Equipment Polymorphic Type - Equipment correctly identifies as Specialized Equipment
[PASS] Lab GPU Detection       - GPU capability verified

>>> [3/4] Running Single-Threaded Booking Lifecycle Tests...
[PASS] Initial Booking Success - First booking should confirm
[PASS] Duplicate Booking Reject - Duplicate booking rejected: Double-booking prevented
[PASS] Reservation Cancellation - Cancellation should succeed and free slot
[PASS] Re-booking Freed Slot   - Slot can be booked after cancellation

>>> [4/4] Running Concurrency & Race-Condition Stress Test (10 Threads)...
[WINNER] Competitor Student secured reservation successfully
[PASS] Exact Single Winner     - Expected exactly 1 successful booking, got: 1
[PASS] Contenders Rejected    - Expected 9 conflicts prevented, got: 9
[PASS] Mutex Lock Registry Active - SlotLockManager recorded thread-safe acquisitions
Concurrency Telemetry: 10 Locks Acquired, 0 Contention Collisions Prevented.

-----
TEST EXECUTION SUMMARY: 14 PASSED, 0 FAILED (100% SUCCESS)
-----
```

9. Challenges Faced & Engineering Solutions

Race Conditions Under Load: Simultaneous threads checked slot availability prior to file write commit, causing double-bookings. *Solution:* Implemented SlotLockManager using fair ReentrantLock instances, re-checking slot occupancy inside the critical section before saving.

Memory Leaks from Idle Locks: A growing lock map could consume unneeded memory over time. *Solution:* Implemented dynamic pruning in the finally block, removing lock entries when no queued threads remain.

Cross-Platform UI Compatibility: Terminal consoles vary in ANSI escape interpretation across OS environments. *Solution:* Implemented robust terminal formatting fallbacks and provided a full Swing desktop GUI alternative.

10. Learnings & Key Takeaways

- Hands-on mastery of Java concurrency primitives (ReentrantLock, ConcurrentHashMap, Callable, Future, ExecutorService).

- Application of SOLID architectural design principles to ensure clear separation between presentation, business rules, and storage.
- Defensive programming practices ensuring data persistence resilience and clean exception recovery.

11. References

- Schildt, Herbert. *Java: The Complete Reference (12th Edition)*. McGraw-Hill Education, 2021.
- Goetz, Brian, et al. *Java Concurrency in Practice*. Addison-Wesley Professional, 2006.
- Bloch, Joshua. *Effective Java (3rd Edition)*. Addison-Wesley, 2018.
- Oracle Corporation. *Java SE Platform Documentation*. <https://docs.oracle.com/en/java/>
- VIT Bhopal University. *CSE2006 Programming in Java Course Syllabus*, 2025–2026.